

SOLID es un acrónimo de conceptos a seguir, siendo guías no reglas.

S - Single Responsibility Principle : Un objeto solo tiene que ser responsable de una responsabilidad.

O - Open/Closed Principle : Entidades de software deberían ser abiertas a extensión pero cerradas a modificación.

L - Liskov Substitution Principle : Un objeto puede ser reemplazado por instancias de su subtipos sin alterar la correctitud del programa, o en otras palabras computación.

I - Interface Segregation Principle : Muchas interfaces definiendo cosas específicas son mejores que algunas generales.

D - Dependency Inversion Principle : Se debería depender de abstracciones y no concretizaciones.

1. Las funciones se acoplan, pueden afectar o hasta inhibir la capacidad de la clase para cumplir con el resto de las funciones y una clase con alto acoplamiento produce diseños frágiles que se rompen de maneras inesperadas cuando se cambia algo, como dice en la página 12. Además el hecho de estar calculando una banda de cosas con las funciones puede llegar a hacer que vaya más lento.
2. Utilizando el principio de open/closed, pudiendo extender el módulo sin borrar o modificar nada como se pide. (página 24 primer PDF)
3. Un código "limpio" es una que es fácil de manejar y mantener, mayormente se puede cumplir esto siguiendo los principios SOLID. Un acoplamiento bajo es vital ya que no sufre de diseños frágiles que pueden romperse fácilmente al cambiar cosas. Mientras que la cohesión es la medida en que los comportamientos de los objetos se relacionan, la cohesión baja es cuando realiza muchas tareas de diverso índole, causando que vaya más lento y tenga más información que procesar. (página 12 primer PDF y página 25 segundo PDF)